

LAPORAN PRAKTIKUM
STRUKTUR DATA
“ BINARY TREE & GRAPH”



disusun Oleh:
FILZI JELILA INDA ROBBANI
(2511533019)

Dosen Pengampu:
Dr. WAHYUDI, S.T, M.T
Asisten Praktikum:
ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadiran Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Struktur Data kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini saya susun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Struktur Data di Universitas Andalas. Laporan ini diharapkan dapat menambah wawasan struktur data Binary Tree dan Graph pada Java .

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan.....	4
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Langkah Kerja Praktikum	6
2.1.1 Membuat kelas pertama (Node).....	6
2.1.2 Membuat kelas kedua (Binary Tree(BTree))	8
2.1.3 Membuat program ketiga (BTreeDriver)	10
2.1.4 Membuat program keempat Graph Traversal.....	12
2.2 Tugas Praktikum	15
2.2.1 Deskripsi Tugas	15
2.2.2 Kelas Node	16
2.2.3 Kelas Graph.....	16
2.2.4 Kelas GUI.....	20
BAB III PENUTUP.....	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya telah dipelajari algoritma sorting, yaitu Insertion Sort, Selection Sort, Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort. Algoritma-algoritma tersebut memiliki mekanisme pengurutan yang berbeda-beda dalam mengolah data. Setelah memahami pengolahan data menggunakan struktur data linear dan algoritma sorting, pada praktikum kali ini pembahasan berlanjut pada struktur data non-linear, yaitu Tree dan Graph.

Tree merupakan struktur data yang digunakan untuk merepresentasikan hubungan hierarki antar data, sedangkan Graph digunakan untuk menggambarkan hubungan antar objek yang lebih kompleks. Pada struktur Tree dikenal konsep root, parent, child, dan leaf node yang membentuk suatu pohon. Sementara itu, pada Graph terdapat simpul (vertex) dan sisi (edge) yang menghubungkan antar simpul.

Selain mempelajari struktur Tree dan Graph, pada praktikum ini juga dipelajari algoritma traversal atau penelusuran data. Traversal pada Binary Tree dilakukan menggunakan metode Inorder, Preorder, dan Postorder. Sedangkan pada Graph digunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian dan penelusuran simpul.

Oleh karena itu, praktikum ini dilakukan untuk memahami konsep, implementasi, serta cara kerja struktur data Tree dan Graph beserta algoritma traversal yang digunakan dalam proses penelusuran data secara efisien.

1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

- a. Memahami konsep Binary Tree dan Graph.
- b. Mengimplementasikan struktur Binary Tree menggunakan Java.
- c. Memahami proses traversal Inorder, Preorder, dan Postorder.
- d. Mengimplementasikan algoritma DFS (Depth First Search).
- e. Mengimplementasikan algoritma BFS (Breadth First Search).
- f. Mengetahui perbedaan cara kerja BFS dan DFS dalam penelusuran graph.
- g. Mampu menampilkan hasil traversal Tree dan Graph menggunakan program Java.

1.3 Manfaat

- a. Menambah pemahaman mengenai struktur data Tree dan Graph.
- b. Melatih kemampuan pemrograman berorientasi objek menggunakan Java.
- c. Memahami penerapan algoritma traversal pada struktur data non-linear.
- d. Mengetahui penggunaan BFS dan DFS dalam pencarian jalur.
- e. Menjadi dasar untuk mempelajari algoritma yang lebih kompleks pada bidang kecerdasan buatan dan jaringan komputer.

BAB II

PEMBAHASAN

2.1 Langkah Kerja Praktikum

2.1.1 Membuat kelas pertama (Node)

- 1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah package baru dengan nama pekan9_nim terus pilih “New”, pilih class. Buat nama class “Node_2511533019”.
- 2) Lalu, masukkan syntax seperti berikut.

```
1) package pekan9_2511533019;
2)
3) public class Node_2511533019 {
4)     int data_3019;
5)     Node_2511533019 left_3019;
6)     Node_2511533019 right_3019;
7)     public Node_2511533019 (int data_3019) {
8)         this.data_3019 = data_3019;
9)         left_3019 = null;
10)        right_3019 = null;
11)    }
12)    public void setleft_3019 (Node_2511533019 node_3019)
13)    {
14)        if (left_3019 == null)
15)            left_3019 = node_3019;
16)    }
17)    public void setRight_3019 (Node_2511533019
18)    node_3019) {
19)        if (right_3019 == null)
20)            right_3019 = node_3019;
21)    }
22)    public Node_2511533019 getleft_3019() {
23)        return left_3019;
24)    }
25)    public Node_2511533019 getRight_3019 () {
26)        return right_3019;
27)    }
28)    public int getData_3019() {
29)        return data_3019;
30)    }
31)    public void setData_3019(int data_3019) {
32)        this.data_3019 = data_3019;
33)    }
34)    void printPreorder_3019 (Node_2511533019 node_3019)
35)    {
36)        if (node_3019 == null)
37)            return;
38)    }
```

```

36)         System.out.print(node_3019.data_3019 +
37)         " ");
38)         printPreorder_3019(node_3019.left_3019);
39)         printPreorder_3019(node_3019.right_3019);
40)     }
41)     void printPostorder_3019(Node_2511533019 node_3019)
42)     {
43)         if (node_3019 == null)
44)             return;
45)         printPostorder_3019 (node_3019.left_3019);
46)         printPostorder_3019 (node_3019.right_3019);
47)         System.out.print(node_3019.data_3019 + " ");
48)     }
49)     void printInorder_3019(Node_2511533019 node_3019) {
50)         if (node_3019== null)
51)             return;
52)         printInorder_3019(node_3019.left_3019);
53)         System.out.print(node_3019.data_3019 + " ");
54)         printInorder_3019(node_3019.right_3019);
55)     }
56)     public String print_3019() {
57)         return this.print_3019("", true, "");
58)     }
59)     public String print_3019(String prefix_3019, boolean
60)     isTail_3019, String sb_3019) {
61)         if (right_3019!= null) {
62)             right_3019.print_3019(prefix_3019 +
63)             (isTail_3019 ? "|" : " "), false, sb_3019);
64)         }
65)         System.out.println( prefix_3019 +
66)         (isTail_3019 ? "\\--" : "/-- ") + data_3019);
67)         if (left_3019 != null) {
68)             left_3019.print_3019 (prefix_3019+
69)             (isTail_3019 ? " ":"|"), true, sb_3019);
70)         }
71)         return sb_3019;
72)     }
73) }

```

Program pertama yaitu Node_2511533019 digunakan untuk merepresentasikan sebuah simpul (node) pada struktur data Binary Tree. Pada class ini terdapat atribut data_3019 untuk menyimpan nilai data bertipe integer, serta atribut left_3019 dan right_3019 yang digunakan untuk menyimpan referensi anak kiri dan anak kanan dari suatu node.

Class ini juga menyediakan method setleft_3019() dan setRight_3019() untuk menghubungkan node dengan childnya, serta method getter untuk mengambil data dan referensi child node. Selain

itu, class ini memiliki method traversal yaitu `printInorder_3019()`, `printPreorder_3019()`, dan `printPostorder_3019()` yang digunakan untuk menelusuri seluruh node pada pohon sesuai urutan traversal yang dipilih. Method `print_3019()` digunakan untuk menampilkan struktur Binary Tree dalam bentuk visual menyerupai pohon.

2.1.2 Membuat kelas kedua (Binary Tree(BTree))

- 1) Klik kanan pada package `Pekan9_nim` terus pilih “New”, pilih class. Buat nama class “`BTree_2511533019`”.
- 2) Masukkan syntax seperti berikut.

```
1) package pekan9_2511533019;
2)
3) public class BTree_2511533019 {
4)     private Node_2511533019 root_3019;
5)     private Node_2511533019 currentNode_3019;
6)     public BTree_2511533019 () {
7)         root_3019 = null;
8)     }
9)     public boolean search_3019(int data_3019) {
10)         return search_3019(root_3019, data_3019);
11)     }
12)     private boolean search_3019(Node_2511533019
node_3019, int data_3019) {
13)         if (node_3019.getData_3019() == data_3019)
14)             return true;
15)         if (node_3019.getleft_3019() != null)
16)             if
17)                 (search_3019(node_3019.getleft_3019(), data_3019))
18)                     return true;
19)         if (node_3019.getRight_3019() != null)
20)             if
21)                 (search_3019(node_3019.getRight_3019(), data_3019))
22)                     return true;
23)         return false;
24)     }
25)     public void printInorder_3019() {
26)         root_3019.printInorder_3019(root_3019);
27)     }
28)     public void printPreOrder_3019() {
29)         root_3019.printPreorder_3019 (root_3019);
30)     }
31)     public void printPostorder_3019() {
32)         root_3019.printPostorder_3019(root_3019);
33)     }
34)     public Node_2511533019 getRoot_3019() {
35)         return root_3019;
36)     }
37)     public boolean isEmpty_3019() {
38)         return root_3019 == null;
39)     }
40) }
```



```

37) }
38) public int countNodes_3019() {
39)     return countNodes_3019(root_3019);
40) }
41)
42) private int countNodes_3019(Node_2511533019
    node_3019) {
43)     int count_3019= 1;
44)     if (node_3019==null) {
45)         return 0;
46)     }else{
47)         count_3019 +=
countNodes_3019(node_3019.getLeft_3019());
48)         count_3019 +=
countNodes_3019(node_3019.getRight_3019());
49)         return count_3019;
50)     }
51) }
52)
53) public void print_3019() {
54)     root_3019.print_3019();
55) }
56) public Node_2511533019 getCurrent_3019() {
57)     return currentNode_3019;
58) }
59) public void setCurrent_3019(Node_2511533019
    node_3019) {
60)     this.currentNode_3019=node_3019;
61) }
62) public void setRoot_3019(Node_2511533019 root_3019)
{
63)     this.root_3019=root_3019;
64) }
65) }

```

Program kedua BTree_2511533019 digunakan sebagai class utama yang mengelola keseluruhan struktur Binary Tree. Class ini memiliki atribut root_3019 yang berfungsi sebagai akar pohon dan currentNode_3019 yang digunakan untuk menyimpan node yang sedang aktif. Method search_3019() digunakan untuk mencari data tertentu pada pohon secara rekursif, sedangkan method countNodes_3019() digunakan untuk menghitung jumlah seluruh node yang terdapat pada Binary Tree. Selain itu, class ini juga menyediakan method printInorder_3019(), printPreOrder_3019(), dan printPostorder_3019() yang berfungsi menjalankan traversal pohon berdasarkan root yang telah ditentukan. Method setRoot_3019() digunakan untuk menentukan akar pohon,

sedangkan method `print_3019()` digunakan untuk menampilkan struktur Binary Tree yang telah dibentuk.

2.1.3 Membuat program ketiga (BTreeDriver)

- 1) Klik kanan pada package `Pekan9_nim` terus pilih “New”, pilih class. Buat nama class “`BtreeDriver_2511533019`”.”
- 2) Lalu buat/masukkan syntax seperti berikut.

```
1) package pekan9_2511533019;
2)
3) public class BtreeDriver_2511533019 {
4)     public static void main(String[] args) {
5)         // TODO Auto-generated method
6)         //Membuat Pohon
7)         BTree_2511533019 tree_3019 = new
BTree_2511533019();
8)         System.out.print("Jumlah Simpul awal pohon:
");
9)         System.
out.println(tree_3019.countNodes_3019());
10)
11)         //menambahkan simpul data 1
12)         Node_2511533019 root_3019 = new
Node_2511533019(1);
13)         //menjadikan simpul 1 sebagai root
14)         tree_3019.setRoot_3019(root_3019);
15)         System.out.println ("Jumlah simpul jika hanya
ada root");
16)         System. out.println
(tree_3019.countNodes_3019());
17)         Node_2511533019 node2_3019 = new
Node_2511533019(2);
18)         Node_2511533019 node3_3019 = new
Node_2511533019(3);
19)         Node_2511533019 node4_3019 = new
Node_2511533019(4);
20)         Node_2511533019 node5_3019 = new
Node_2511533019(5);
21)         Node_2511533019 node6_3019 = new
Node_2511533019(6);
22)         Node_2511533019 node7_3019 = new
Node_2511533019(7);
23)         Node_2511533019 node8_3019 = new
Node_2511533019(8);
24)         Node_2511533019 node9_3019 = new
Node_2511533019(9);
25)         root_3019.setleft_3019(node2_3019);
26)         node2_3019.setleft_3019(node4_3019);
27)         node2_3019.setRight_3019(node5_3019);
28)         node4_3019.setRight_3019(node8_3019);
29)         root_3019.setRight_3019(node3_3019);
30)         node3_3019.setleft_3019(node6_3019);
```

```

31)         node3_3019.setRight_3019(node7_3019);
32)         node6_3019.setleft_3019(node9_3019);
33)         //set root
34)
35)         tree_3019.setCurrent_3019(tree_3019.getRoot_3019());
36)         System.out.println("menampilkan simpul
           terakhir : ");
37)         System.out.println(tree_3019.getCurrent_3019().getDa
           ta_3019());
38)         System.out.println("Jumlah simpul; setelah
           simpul 7 ditambahkan" ) ;
39)         System.out.println(tree_3019.countNodes_3019());
40)         System.out.println("InOrder: ");
41)         tree_3019.printInorder_3019();
42)         System.out.println("\nPreorder: ");
43)         tree_3019.printPreOrder_3019();
44)         System.out.println (" \nPostorder : ");
45)         tree_3019.printPostorder_3019();
46)         System.out.println("\nDmenampilkan simpul
           dalam bentuk pohon");
47)         tree_3019.print_3019() ;
48)     }
49) }

```

Program ketiga yaitu BtreeDriver_2511533019 berfungsi sebagai driver program atau class utama yang digunakan untuk menjalankan implementasi Binary Tree.

Pada class ini dibuat objek Binary Tree serta beberapa node yang berisi data dari 1 sampai 9. Selanjutnya setiap node dihubungkan menggunakan method setleft_3019() dan setRight_3019() sehingga membentuk struktur pohon biner. Setelah pohon selesai dibuat, program menampilkan jumlah node yang ada pada pohon menggunakan method countNodes_3019(), kemudian melakukan traversal menggunakan metode Inorder, Preorder, dan Postorder. Selain itu, program juga menampilkan struktur Binary Tree secara visual sehingga hubungan antar node dapat terlihat dengan lebih jelas.

- 3) Run program hingga keluar output seperti berikut, apabila tidak seperti pada gambar, cek kesalahan pada syntax/ kekurangan syntaxnya, dan perbaiki.

```

Console X
<terminated> BtreeDriver_2511533019 [Java Application] C:\Users
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir :
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder :
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|
| /-- 7
| /-- 3
| | \--6
| | \--9
|--1
| /-- 5
|--2
| /-- 8
|--4

```

2.1.4 Membuat program keempat Graph Traversal

- 1) Klik kanan package pekan9_nim dan pilih “New”, pilih others, scroll kebawah dan klik JFram pada Swing Designer. Lalu next bikin nama “GraphTraversal_2511533019”.
- 2) Tambahkan syntax seperti berikut untuk membangun GUI sesuai dengan tampilan yang diinginkan serta logika sorting yang akan digunakan.

```

1) package pekan9_2511533019;
2)
3) import java.util.*;
4)
5) public class GraphTraversal_2511533019 {
6)     private Map<String, List<String>> graph_3019 = new
HashMap<>();
7)
8)     // Menambahkan edge (graf tak berarah)
9)     public void addEdge_3019(String node1_3019,
String node2_3019) {
10)         graph_3019.putIfAbsent(node1_3019, new
ArrayList<>());
11)         graph_3019.putIfAbsent(node2_3019, new
ArrayList<>());
12)         graph_3019.get(node1_3019).add(node2_3019);

```

```

13)         graph_3019.get(node2_3019).add(node1_3019);
14)     }
15)
16)     // Menampilkan graf awal
17)     public void printGraph_3019() {
18)         System.out.println("Graf Awal (Adjacency
19)         List):");
20)         for (String node_3019 :
21)         graph_3019.keySet()) {
22)             System.out.print(node_3019 + " -> ");
23)             List<String> neighbors_3019 =
24)             graph_3019.get(node_3019);
25)             System.out.println(String.join(", ",
26)             neighbors_3019));
27)         }
28)         System.out.println();
29)     }
30)
31)     // DFS rekursif
32)     public void dfs_3019(String start_3019) {
33)         Set<String> visited_3019 = new HashSet<>();
34)         System.out.println("Penelusuran DFS:");
35)         dfsHelper_3019(start_3019, visited_3019);
36)         System.out.println();
37)     }
38)
39)     private void dfsHelper_3019(String
40)     current_3019, Set<String> visited_3019) {
41)         if (visited_3019.contains(current_3019))
42)             return;
43)         visited_3019.add(current_3019);
44)         System.out.print(current_3019 + " ");
45)         for (String neighbor_3019 :
46)         graph_3019.getOrDefault(current_3019, new
47)         ArrayList<>())) {
48)             dfsHelper_3019(neighbor_3019,
49)             visited_3019);
50)         }
51)     }
52)
53)     // BFS iteratif
54)     public void bfs_3019(String start_3019) {
55)         Set<String> visited_3019 = new HashSet<>();
56)         Queue<String> queue_3019 = new LinkedList<>();
57)
58)         queue_3019.add(start_3019);
59)         visited_3019.add(start_3019);
60)
61)         System.out.println("Penelusuran BFS:");
62)         while (!queue_3019.isEmpty()) {
63)             String current_3019 = queue_3019.poll();
64)             System.out.print(current_3019 + " ");
65)
66)             for (String neighbor :
67)             graph_3019.getOrDefault(current_3019, new
68)             ArrayList<>())) {

```

```

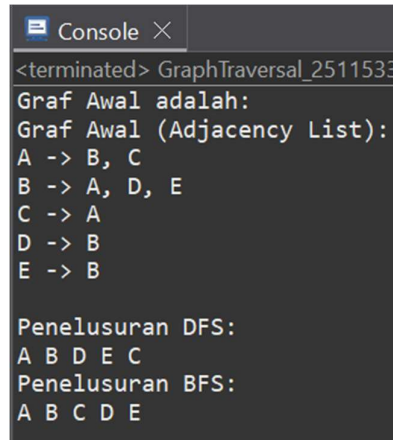
58)         if (!visited_3019.contains(neighbor))
59)         {
60)             queue_3019.add(neighbor);
61)             visited_3019.add(neighbor);
62)         }
63)     }
64)     System.out.println();
65) }
66)
67) // Main
68) public static void main(String[] args) {
69)     GraphTraversal_2511533019 graph_3019 = new
70)     GraphTraversal_2511533019();
71)
72)     // Contoh graf: A-B, A-C, B-D, B-E
73)     graph_3019.addEdge_3019("A", "B");
74)     graph_3019.addEdge_3019("A", "C");
75)     graph_3019.addEdge_3019("B", "D");
76)     graph_3019.addEdge_3019("B", "E");
77)
78)     // Cetak graf awal
79)     System.out.println("Graf Awal adalah: ");
80)     graph_3019.printGraph_3019();
81)
82)     // Lakukan penelusuran
83)     graph_3019.dfs_3019("A");
84)     graph_3019.bfs_3019("A");
85) }
86) }

```

Program keempat yaitu GraphTraversal_2511533019 digunakan untuk mengimplementasikan struktur data Graph beserta algoritma penelusuran Depth First Search (DFS) dan Breadth First Search (BFS). Graph direpresentasikan menggunakan HashMap yang menyimpan daftar tetangga dari setiap simpul dalam bentuk adjacency list. Method addEdge_3019() digunakan untuk menambahkan hubungan antar simpul pada graph, sedangkan method printGraph_3019() digunakan untuk menampilkan seluruh hubungan yang ada pada graph. Method dfs_3019() melakukan penelusuran secara mendalam menggunakan pendekatan rekursif dengan bantuan method dfsHelper_3019(), sedangkan method bfs_3019() melakukan penelusuran secara melebar menggunakan struktur data Queue. Pada method main(), dibuat contoh graph sederhana yang terdiri dari beberapa simpul dan sisi, kemudian dilakukan penelusuran

menggunakan algoritma DFS dan BFS untuk memperlihatkan perbedaan urutan kunjungan simpul pada kedua algoritma tersebut.

- 3) Run program hingga mengeluarkan tampilan seperti berikut.



```
<terminated> GraphTraversal_2511533
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

2.2 Tugas Praktikum

2.2.1 Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph. Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain.

Peta dapat berupa: Gedung Perkuliahan, Fakultas, Universitas Sekolah, Rumah Sakit, Mall, Tempat Wisata, Bandara Atau lokasi lainnya.

Program harus mampu: Menampilkan graph dalam bentuk visual, Menentukan titik awal Start, Menentukan titik tujuan Goal, Melakukan pencarian menggunakan BFS, Melakukan pencarian menggunakan DFS, Menampilkan jalur yang ditemukan, Menampilkan urutan simpul yang dikunjungi, Urutan node yang dikunjungi, dan Jumlah node yang dieksplorasi.

2.2.2 Kelas Node

- 1) Klik kanan package Pekan9 dan pilih “New”, pilih class. Lalu buat nama “NodeRS_2511533019”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package pekan9_2511533019;
2)
3) public class NodeRS_2511533019 {
4)
5)     private String nama_3019;
6)     private int x_3019;
7)     private int y_3019;
8)
9)     public NodeRS_2511533019(String nama_3019, int
x_3019, int y_3019) {
10)         this.nama_3019 = nama_3019;
11)         this.x_3019 = x_3019;
12)         this.y_3019 = y_3019;
13)     }
14)     public String getNama_3019() {
15)         return nama_3019;
16)     }
17)     public int getX_3019() {
18)         return x_3019;
19)     }
20)     public int getY_3019() {
21)         return y_3019;
22)     }
23) }
```

Program ini dibuat merepresentasikan sebuah node atau lokasi pada graph rumah sakit. Setiap node memiliki nama lokasi serta koordinat x dan y yang dapat digunakan untuk menentukan posisi node pada tampilan visual graph.

2.2.3 Kelas Graph

- 1) Klik kanan package Pekan9 dan pilih “New”, pilih class. Lalu buat nama “GraphRS_2511533019”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package pekan9_2511533019;
2)
3) import java.util.*;
4)
5) public class GraphRS_2511533019 {
6)
7)     private Map<String, List<String>> graph_3019;
8)     private List<String> visitedOrder_3019;
9)     private List<String> path_3019;
```



```

10)
11)     public GraphRS_2511533019() {
12)         graph_3019 = new HashMap<>();
13)         visitedOrder_3019 = new ArrayList<>();
14)         path_3019 = new ArrayList<>();
15)         // EDGE RUMAH SAKIT
16)         addEdge_3019("Parkir", "Pintu Masuk");
17)         addEdge_3019("Parkir", "Kantin");
18)         addEdge_3019("Pintu Masuk", "IGD");
19)         addEdge_3019("Pintu Masuk", "Apotek");
20)         addEdge_3019("IGD", "Radiologi");
21)         addEdge_3019("IGD", "Laboratorium");
22)         addEdge_3019("Apotek", "Kantin");
23)         addEdge_3019("Apotek", "Laboratorium");
24)         addEdge_3019("Apotek", "Poliklinik");
25)         addEdge_3019("Laboratorium", "ICU");
26)         addEdge_3019("Laboratorium", "Ruang
Rawat");
27)         addEdge_3019("Kantin", "Poliklinik");
28)         addEdge_3019("Poliklinik", "Ruang Rawat");
29)         addEdge_3019("Ruang Rawat", "ICU");
30)     }
31)     public void addEdge_3019(String node1_3019,
String node2_3019) {
32)         graph_3019.putIfAbsent(node1_3019, new
ArrayList<>());
33)         graph_3019.putIfAbsent(node2_3019, new
ArrayList<>());
34)         graph_3019.get(node1_3019).add(node2_3019);
35)         graph_3019.get(node2_3019).add(node1_3019);
36)     }
37)     public List<String> getVisitedOrder_3019() {
38)         return visitedOrder_3019;
39)     }
40)     public List<String> getPath_3019() {
41)         return path_3019;
42)     }
43)     public int getExploredCount_3019() {
44)         return visitedOrder_3019.size();
45)     }
46)     public void BFS_3019(String start_3019, String
goal_3019) {
47)         visitedOrder_3019.clear();
48)         path_3019.clear();
49)         Queue<String> queue_3019 = new
LinkedList<>();
50)         Set<String> visited_3019 = new HashSet<>();
51)         Map<String, String> parent_3019 = new
HashMap<>();
52)         queue_3019.add(start_3019);

```

```

53)         visited_3019.add(start_3019);
54)         while (!queue_3019.isEmpty()) {
55)             String current_3019 =
queue_3019.poll();
56)             visitedOrder_3019.add(current_3019);
57)             if (current_3019.equals(goal_3019))
58)                 break;
59)             for (String neighbor_3019 :
graph_3019.get(current_3019)) {
60)                 if
(!visited_3019.contains(neighbor_3019)) {
61)                     visited_3019.add(neighbor_3019);
62)                     parent_3019.put(neighbor_3019,
current_3019);
63)                     queue_3019.add(neighbor_3019);
64)                 }
65)             }
66)         }
67)         buildPath_3019(parent_3019, start_3019,
goal_3019);
68)     }
69)     public void DFS_3019(String start_3019, String
goal_3019) {
70)         visitedOrder_3019.clear();
71)         path_3019.clear();
72)         Stack<String> stack_3019 = new Stack<>();
73)         Set<String> visited_3019 = new HashSet<>();
74)         Map<String, String> parent_3019 = new
HashMap<>();
75)         stack_3019.push(start_3019);
76)         while (!stack_3019.isEmpty()) {
77)             String current_3019 = stack_3019.pop();
78)             if
(!visited_3019.contains(current_3019)) {
79)                 visited_3019.add(current_3019);
80)                 visitedOrder_3019.add(current_3019);
81)                 if (current_3019.equals(goal_3019))
82)                     break;
83)                 List<String> neighbors_3019 = new
ArrayList<>(graph_3019.get(current_3019));
84)                 Collections.reverse(neighbors_3019);
85)                 for (String neighbor_3019 :
neighbors_3019) {
86)                     if
(!visited_3019.contains(neighbor_3019)) {
87)                         parent_3019.put(neighbor_3019, current_3019);

```

```

88)         stack_3019.push(neighbor_3019);
89)     }
90) }
91) }
92) }
93)     buildPath_3019(parent_3019, start_3019,
goal_3019);
94) }
95)     private void buildPath_3019(
96)         Map<String, String> parent_3019,
97)         String start_3019,
98)         String goal_3019) {
99)
100)         String current_3019 = goal_3019;
101)         while (current_3019 != null) {
102)             path_3019.add(current_3019);
103)             if
104)                 (current_3019.equals(start_3019))
105)                 break;
106)             current_3019 =
107)                 parent_3019.get(current_3019);
108)         }
109)         Collections.reverse(path_3019);
110)     }
111)     public void resetGraph_3019() {
112)         visitedOrder_3019.clear();
113)         path_3019.clear();
114)     }

```

Kelas GraphRS_2511533019 digunakan untuk membangun struktur graph rumah sakit serta mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search). Graph disimpan menggunakan struktur data Map<String, List<String>> sebagai adjacency list yang menyimpan hubungan antar lokasi. Constructor berfungsi membuat graph dan menambahkan seluruh edge rumah sakit. Method addEdge_3019() digunakan untuk menghubungkan dua node secara dua arah. Method BFS_3019() melakukan pencarian jalur menggunakan Queue dengan menelusuri node per level hingga menemukan tujuan, sedangkan DFS_3019() menggunakan Stack untuk menelusuri node secara mendalam terlebih dahulu. Selama proses pencarian, urutan node yang

dikunjungi disimpan pada `visitedOrder_3019` dan jalur yang ditemukan dibentuk menggunakan method `buildPath_3019()` berdasarkan parent node yang telah disimpan. Method `resetGraph_3019()` digunakan untuk menghapus hasil pencarian sebelumnya.

2.2.4 Kelas GUI

- 1) Klik kanan package `Pekan9` dan pilih “New”, pilih class. Lalu buat nama “`GUIRumahSakit_2511533019`”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package pekan9_2511533019;
2)
3) import java.awt.Color;
4) import java.awt.EventQueue;
5) import java.awt.Font;
6)
7) import javax.swing.DefaultComboBoxModel;
8) import javax.swing.JButton;
9) import javax.swing.JComboBox;
10) import javax.swing.JFrame;
11) import javax.swing.JLabel;
12) import javax.swing.JPanel;
13) import javax.swing.JTextArea;
14) import javax.swing.JTextField;
15) import javax.swing.SwingConstants;
16) import javax.swing.border.EmptyBorder;
17)
18) public class GUIRumahSakit_2511533019 extends
    JFrame {
19)
20)     private static final long serialVersionUID =
        1L;
21)     private JPanel contentPane;
22)     private JTextField txtJudul_3019;
23)     private JComboBox<String> comboAwal_3019;
24)     private JComboBox<String> comboTujuan_3019;
25)     private JTextArea txtGraph_3019;
26)     private JTextArea txtHasil_3019;
27)     private GraphRS_2511533019 graph_3019 = new
        GraphRS_2511533019();
28)     public static void main(String[] args) {
29)         EventQueue.invokeLater(new Runnable() {
30)             public void run() {
31)                 try {
32)                     GUIRumahSakit_2511533019 frame
= new GUIRumahSakit_2511533019();
33)                     frame.setVisible(true);
34)                 } catch (Exception e) {
35)                     e.printStackTrace();
36)                 }
            }
        }
    }
}
```

```

37)         }
38)     });
39) }
40) public GUIRumahSakit_2511533019() {
41)     setTitle("BFS dan DFS Rumah Sakit");
42)
43)     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
44)     setBounds(100, 100, 815, 420);
45)     contentPane = new JPanel();
46)     contentPane.setBorder(new EmptyBorder(5, 5,
47) 5, 5));
48)     setContentPane(contentPane);
49)     contentPane.setLayout(null);
50)
51)     txtJudul_3019 = new JTextField();
52)     txtJudul_3019.setText("PENCARIAN JALUR
53) RUMAH SAKIT MENGGUNAKAN BFS DAN DFS");
54)
55)     txtJudul_3019.setHorizontalAlignment(SwingConstants
56) .CENTER);
57)     txtJudul_3019.setForeground(Color.WHITE);
58)     txtJudul_3019.setBackground(new Color(0,
59) 64, 128));
60)     txtJudul_3019.setFont(new Font("Tw Cen MT",
61) Font.BOLD, 13));
62)     txtJudul_3019.setEditable(false);
63)     txtJudul_3019.setBounds(0, 0, 801, 30);
64)     contentPane.add(txtJudul_3019);
65)
66)     JLabel lblAwal_3019 = new JLabel("Lokasi
67) Awal :");
68)     lblAwal_3019.setBounds(10, 40, 90, 20);
69)     contentPane.add(lblAwal_3019);
70)
71)     JLabel lblTujuan_3019 = new JLabel("Lokasi
72) Tujuan :");
73)     lblTujuan_3019.setBounds(10, 65, 90, 20);
74)     contentPane.add(lblTujuan_3019);
75)
76)     comboAwal_3019 = new JComboBox<>();
77)     comboAwal_3019.setModel(new
78) DefaultComboBoxModel<>(new String[] {
79)         "Pintu Masuk",
80)         "IGD",
81)         "Apotek",
82)         "Poliklinik",
83)         "Laboratorium",
84)         "Radiologi",
85)         "Ruang Rawat",
86)         "ICU",
87)         "Kantin",
88)         "Parkir"
89)     });
90)     comboAwal_3019.setBounds(110, 40, 140, 22);
91)     contentPane.add(comboAwal_3019);
92)

```

```

83)         comboTujuan_3019 = new JComboBox<>();
84)         comboTujuan_3019.setModel(new
DefaultComboBoxModel<>(new String[] {
85)             "Pintu Masuk",
86)             "IGD",
87)             "Apotek",
88)             "Poliklinik",
89)             "Laboratorium",
90)             "Radiologi",
91)             "Ruang Rawat",
92)             "ICU",
93)             "Kantin",
94)             "Parkir"
95)         }));
96)         comboTujuan_3019.setBounds(110, 65, 140,
22);
97)         contentPane.add(comboTujuan_3019);
98)
99)         JButton btnBFS_3019 = new JButton("BFS");
100)        btnBFS_3019.setBackground(new
Color(255, 255, 128));
101)        btnBFS_3019.setBounds(504, 41, 80,
25);
102)        contentPane.add(btnBFS_3019);
103)
104)        JButton btnDFS_3019 = new
JButton("DFS");
105)        btnDFS_3019.setBackground(new
Color(255, 128, 128));
106)        btnDFS_3019.setBounds(601, 41, 80,
25);
107)        contentPane.add(btnDFS_3019);
108)
109)        JButton btnReset_3019 = new
JButton("RESET");
110)        btnReset_3019.setForeground(Color.WHITE);
111)        btnReset_3019.setBackground(new
Color(128, 0, 64));
112)        btnReset_3019.setBounds(691, 41, 100,
25);
113)        contentPane.add(btnReset_3019);
114)
115)        txtGraph_3019 = new JTextArea();
116)        txtGraph_3019.setEditable(false);
117)        txtGraph_3019.setFont(new
Font("Monospaced", Font.PLAIN, 12));
118)
119)        txtGraph_3019.setText(
120)            "STRUKTUR GRAPH RUMAH
SAKIT\r\n\r\nParkir ---- Pintu Masuk ---- IGD ----
Radiologi\r\n    |                      |                      |\r\n
|                      |                      |\r\nKantin ---- Apotek
---- Laboratorium ---- ICU\r\n    |                      |
|\r\n    |\r\n    |                      |                      |
|\r\n    +----- Poliklinik ---- Ruang Rawat --+"

```

```

121)                );
122)
123)                txtGraph_3019.setBounds(120, 99, 560,
124)                157);
125)                contentPane.add(txtGraph_3019);
126)                JLabel lblHasil_3019 = new
127)                JLabel("Hasil Pencarian");
128)                lblHasil_3019.setFont(new Font("Tw
129)                Cen MT", Font.BOLD, 13));
130)                lblHasil_3019.setBounds(10, 250, 200,
131)                20);
132)                contentPane.add(lblHasil_3019);
133)                txtHasil_3019 = new JTextArea();
134)                txtHasil_3019.setEditable(false);
135)                txtHasil_3019.setFont(new Font("Tw
136)                Cen MT", Font.PLAIN, 12));
137)                txtHasil_3019.setText(
138)                "Jalur :\n\n" +
139)                "Node Dikunjungi :\n\n" +
140)                "Jumlah Node :";
141)                txtHasil_3019.setBounds(10, 275, 791,
142)                90);
143)                contentPane.add(txtHasil_3019);
144)                btnBFS_3019.addActionListener(e -> {
145)
146)                String start_3019 =
147)                comboAwal_3019.getSelectedItem().toString();
148)                String goal_3019 =
149)                comboTujuan_3019.getSelectedItem().toString();
150)                graph_3019.BFS_3019(start_3019,
151)                goal_3019);
152)                txtHasil_3019.setText(
153)                "Jalur : " +
154)                String.join(" -> ", graph_3019.getPath_3019())
155)                + "\n\nNode
156)                Dikunjungi : " + String.join(" ->
157)                ", graph_3019.getVisitedOrder_3019())
158)                + "\n\nJumlah
159)                Node : " + graph_3019.getExploredCount_3019()
160)                );
161)                });
162)                btnDFS_3019.addActionListener(e -> {
163)                String start_3019 =
164)                comboAwal_3019.getSelectedItem().toString();
165)                String goal_3019 =
166)                comboTujuan_3019.getSelectedItem().toString();
167)                graph_3019.DFS_3019(start_3019,
168)                goal_3019);
169)                txtHasil_3019.setText(

```

```

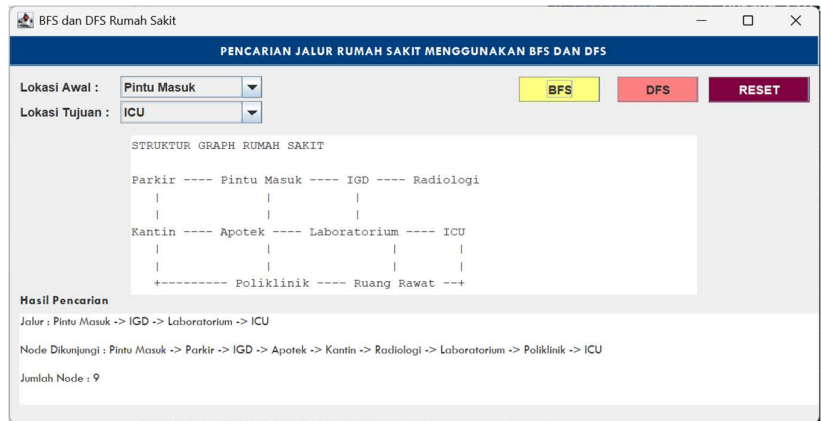
160)         "Jalur : "+ String.join("
-> ",graph_3019.getPath_3019())
161)         + "\n\nNode
Dikunjungi : " + String.join(" ->
",graph_3019.getVisitedOrder_3019())
162)         + "\n\nJumlah
Node : "+ graph_3019.getExploredCount_3019()
163)     );
164)     });
165)
166)     btnReset_3019.addActionListener(e ->
{
167)         graph_3019.resetGraph_3019();
168)         comboAwal_3019.setSelectedIndex(0);
169)         comboTujuan_3019.setSelectedIndex(0);
170)         txtHasil_3019.setText(
171)             "Jalur :\n\n" +
172)             "Node dikunjungi :\n\n" +
173)             "Jumlah Node :";
174)     });
175) });
176) }
177) }

```

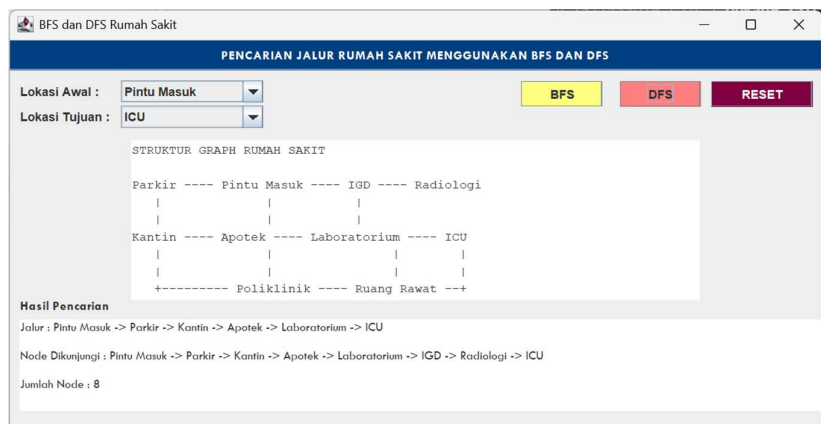
Kelas `GUIRumahSakit_2511533019` merupakan antarmuka pengguna berbasis Java Swing yang digunakan untuk menjalankan pencarian jalur pada graph rumah sakit. Class ini menyediakan dua `JComboBox` untuk memilih lokasi awal dan lokasi tujuan, tombol BFS untuk menjalankan algoritma Breadth First Search, tombol DFS untuk menjalankan algoritma Depth First Search, serta tombol Reset untuk mengembalikan tampilan ke kondisi awal. Selain itu terdapat `JTextArea` yang digunakan untuk menampilkan struktur graph rumah sakit dan `JTextArea` lainnya untuk menampilkan hasil pencarian berupa jalur yang ditemukan, urutan node yang dikunjungi, dan jumlah node yang dieksplorasi. GUI ini berfungsi sebagai penghubung antara pengguna dengan class `GraphRS_2511533019` sehingga proses pencarian jalur dapat dilakukan secara interaktif.

3) Run Program

- BFS :



- **DFS :**



BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Binary Tree merupakan struktur data hierarki yang terdiri atas root, node, dan child node. Traversal pada Binary Tree dapat dilakukan menggunakan metode Inorder, Preorder, dan Postorder untuk mengakses seluruh simpul pada pohon.

Selain itu, Graph digunakan untuk merepresentasikan hubungan antar objek yang lebih kompleks. Algoritma DFS melakukan penelusuran secara mendalam hingga mencapai simpul terakhir sebelum kembali ke simpul sebelumnya, sedangkan BFS melakukan penelusuran secara melebar berdasarkan tingkat kedalaman simpul.

Melalui praktikum ini mahasiswa memperoleh pemahaman mengenai implementasi struktur data non-linear dan algoritma traversal yang menjadi dasar dalam berbagai aplikasi komputasi modern..

DAFTAR PUSTAKA

- [1] Wahyudi, “*Binary Tree & Graph*” PowerPoint slides, Praktikum Struktur Data, Universitas Andalas, Padang, 2025.
- [2] Oracle Corporation, Java Platform, Standard Edition Documentation. Oracle, 2025